22. Observer Design Pattern in Java

22.1 Introduction

The **Observer Design Pattern** is a **behavioral pattern** used to create a one-to-many dependency between objects, so that when one object changes state, all its dependents are notified and updated automatically.

22.2 Problem it Solves

You need a way for objects (observers) to react automatically when the state of another object (subject) changes without tightly coupling them.

22.3 What is Observer Pattern?

It defines:

- A **Subject** that maintains a list of dependents (observers)
- A **notify mechanism** to inform all observers when the state changes

22.4 Real-World Analogy

A **YouTube channel subscriber** gets notifications when a new video is published.

22.5 Structure

```
package com.mannemlokesh;

public interface Observer {
	void update(String message);
}

interface Subject {
	void attach(Observer o);

	void detach(Observer o);

	void notifyObservers();
}
```

22.6 Benefits

- Loose coupling between subject and observers
- Broadcast communication support
- Easy to add new observers

22.7 Implementation Steps

1. Define an Observer interface
2. Create concrete observer classes
3. Define a Subject interface
4. Implement concrete subject and maintain observer list

22.8 Examples

Example1: News Channel Notifier

```
package com.mannemlokesh.example1;

import java.util.ArrayList;
import java.util.List;

interface Observer {
	void update(String news);
}

class NewsChannel implements Observer {
	private String name;

	public NewsChannel(String name) {
		this.name = name;
	}

	public void update(String news) {
		System.out.println(name + " received news: " + news);
	}
}

interface Subject {
	void attach(Observer o);

	void detach(Observer o);

	void notifyObservers();
}

class NewsAgency implements Subject {
	private List<Observer> observers = new ArrayList<>();
	private String news;

	public void setNews(String news) {
		this.news = news;
		notifyObservers();
	}

	public void attach(Observer o) {
		observers.add(o);
	}

	public void detach(Observer o) {
		observers.remove(o);
	}

	public void notifyObservers() {
		for (Observer o : observers) {
			o.update(news);
		}
	}
}

public class Example1NewsClient {
	public static void main(String[] args) {
		NewsAgency agency = new NewsAgency();
		Observer ch1 = new NewsChannel("Channel 1");
		Observer ch2 = new NewsChannel("Channel 2");
```

```
		agency.attach(ch1);
		agency.attach(ch2);

		agency.setNews("Elections are coming!");
	}
}
```

Output:

```
Channel 1 received news: Elections are coming!
Channel 2 received news: Elections are coming!
```

Example2: Stock Market Observer

```
package com.mannemlokesh.example2;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

interface StockObserver {
	void priceUpdated(String stock, double price);
}

class StockDisplay implements StockObserver {
	private String displayId;

	public StockDisplay(String id) {
		this.displayId = id;
	}

	public void priceUpdated(String stock, double price) {
		System.out.println(displayId + " - " + stock + " updated to: $" +
                                                                    price);
	}
}

class StockMarket {
	private Map<String, Double> stocks = new HashMap<>();
	private List<StockObserver> observers = new ArrayList<>();

	public void attach(StockObserver observer) {
		observers.add(observer);
	}

	public void updateStock(String symbol, double price) {
		stocks.put(symbol, price);
		notifyObservers(symbol, price);
	}

	private void notifyObservers(String symbol, double price) {
		for (StockObserver obs : observers) {
			obs.priceUpdated(symbol, price);
		}
	}
}
```

```
public class Example2StockMarketClient {
	public static void main(String[] args) {
		StockMarket market = new StockMarket();
		market.attach(new StockDisplay("Display A"));
		market.attach(new StockDisplay("Display B"));

		market.updateStock("AAPL", 145.30);
		market.updateStock("GOOG", 2730.00);
	}
}
```

Output:

```
Display A - AAPL updated to: $145.3
Display B - AAPL updated to: $145.3
Display A - GOOG updated to: $2730.0
Display B - GOOG updated to: $2730.0
```

Example3: Event-Driven UI Framework

```
package com.mannemlokesh.example3;

import java.util.ArrayList;
import java.util.List;

interface UIListener {
	void onClick(String componentId);
}

class Button {
	private String id;
	private List<UIListener> listeners = new ArrayList<>();

	public Button(String id) {
		this.id = id;
	}

	public void addListener(UIListener listener) {
		listeners.add(listener);
	}

	public void click() {
		System.out.println("Button " + id + " clicked");
		for (UIListener listener : listeners) {
			listener.onClick(id);
		}
	}
}

class DialogBox implements UIListener {
	public void onClick(String componentId) {
		System.out.println("DialogBox: Action triggered by " + componentId);
	}
}

public class Example3UIClient {
	public static void main(String[] args) {
		Button button = new Button("submitBtn");
		DialogBox dialog = new DialogBox();
```

```
            button.addListener(dialog);
            button.click();
        }
}
```

Output:

```
Button submitBtn clicked
DialogBox: Action triggered by submitBtn
```

22.9 Observer vs Other Patterns

Pattern	Purpose
Observer	Notify subscribers on subject changes
Mediator	Encapsulate interaction logic
Command	Encapsulate action as an object

22.10 Pros and Cons

✅ **Pros**

- Promotes loose coupling
- Easy to add/remove observers
- Works well in event-driven systems

❌ **Cons**

- Can cause memory leaks if observers aren't removed
- Notifications may become unmanageable in large systems

22.11 When to Use / Not to Use

Use When:

- You need to broadcast changes to multiple components
- Objects should not know each other directly

Avoid When:

- Few objects involved (overhead not worth it)
- Real-time updates not required

22.12 Summary

Feature	Description
Intent	Notify multiple objects about state changes
Use Case	Event listeners, data binding, messaging
Key Benefit	Decouples subjects and observers

The **Observer Pattern** is perfect for situations where multiple components need to stay in sync with state changes.